

1. Análisis léxico con Flex

El análisis léxico puede modelarse conceptualmente como una función que recorre el código fuente, identifica lexemas y los transforma en tokens. Flex permite automatizar la implementación de esta función a partir de una especificación de patrones léxicos.

Sea C el código fuente y sea T el conjunto de tokens del lenguaje. Podemos representar el analizador léxico como:

$$L : C \rightarrow T^*$$

donde T^* representa una secuencia de tokens.

De forma recursiva, el lexer puede interpretarse como una función que consume progresivamente la entrada. Si c representa la entrada restante, el analizador busca el siguiente lexema reconocido por alguno de los patrones definidos y ejecuta la acción asociada.

$$L(c) = \begin{cases} [] & \text{si } c = \epsilon \\ t :: L(c') & \text{si el lexema } l \text{ coincide con un patrón y} \\ & \text{la acción asociada produce el token } t \\ l :: L(c') & \text{si el lexema } l \text{ no coincide con ningún patrón} \end{cases}$$

donde c' representa la entrada restante después de consumir el lexema l .

1.1. Patrones y acciones

Una especificación de Flex está formada por reglas que relacionan un *patrón* con una *acción*. El comienzo de esta sección se indica mediante `%%`.

Por ejemplo:

```
%%
"username"    printf("%s", getlogin());
```

En esta regla, `username` es el patrón y `printf("%s", getlogin())` es la acción.

Cuando Flex encuentra la cadena `username` en la entrada, ejecuta la acción asociada y reemplaza dicha cadena por el nombre de usuario obtenido mediante `getlogin()`.

Por defecto, cuando Flex encuentra texto que no coincide con ninguno de los patrones definidos, dicho texto se copia directamente a la salida. Por lo tanto, en el ejemplo anterior, el analizador copia el archivo de entrada a la salida, excepto por las apariciones de `username`, que son reemplazadas por el nombre de usuario correspondiente.

Este comportamiento permite entender una regla de Flex como una transformación:

$$\text{patrón léxico} \longrightarrow \text{acción}$$

donde la acción determina qué ocurre cuando el patrón es reconocido.

1.2. Expresiones regulares y patrones de Flex

Los patrones de Flex se construyen principalmente mediante expresiones regulares. Estas permiten describir conjuntos de cadenas que pueden aparecer en el código fuente.

1.2.1. Caracteres literales

Un carácter escrito directamente en el patrón representa ese carácter.

```
+  
-  
*  
=  
;
```

Por ejemplo:

```
"+"    { return SUMA; }  
"="    { return ASIGNACION; }  
";"    { return PUNTO_COMA; }
```

Las comillas permiten representar explícitamente una cadena literal.

1.2.2. Cadenas literales

Una secuencia de caracteres entre comillas representa exactamente esa secuencia.

```
"int"  
"while"  
"return"  
"=="
```

Por ejemplo:

```
"while"    { return WHILE; }  
"return"   { return RETURN; }
```

1.2.3. Clase de caracteres

Los corchetes permiten especificar un conjunto de caracteres. El patrón coincide con exactamente un carácter perteneciente a ese conjunto.

```
[abc]
```

coincide con:

```
a  
b  
c
```

También pueden utilizarse rangos:

```
[a-z]  
[A-Z]  
[0-9]
```

Por ejemplo:

```
[0-9]    { return DIGITO; }
```

reconoce cualquier dígito entre 0 y 9.

Los rangos pueden combinarse:

```
[a-zA-Z]  
[a-zA-Z0-9]
```

1.2.4. Negación de una clase

El carácter `^` como primer carácter dentro de una clase niega la clase.

`[^0-9]`

coincide con cualquier carácter que no sea un dígito.

Por ejemplo:

`[^a-zA-Z0-9]`

coincide con cualquier carácter que no sea una letra ni un número.

1.2.5. Concatenación

Dos patrones escritos uno después del otro representan una concatenación. Ambos deben aparecer consecutivamente en la entrada.

Por ejemplo:

`ab`

coincide con:

`ab`

pero no con:

`a`

`b`

`ac`

Un identificador puede expresarse mediante:

`[a-zA-Z_][a-zA-Z0-9_]*`

donde primero debe aparecer una letra o guion bajo y posteriormente pueden aparecer letras, números o guiones bajos.

1.2.6. Alternancia

El operador `|` representa una alternativa. El patrón coincide con una opción u otra.

`if|while|for`

coincide con cualquiera de:

`if`

`while`

`for`

También puede utilizarse con grupos:

`(if|while|for)`

1.2.7. Cero o más repeticiones

El operador `*` significa que el elemento anterior puede aparecer cero o más veces.

`[0-9]*`

coincide con:

```
" "  
"1 "  
"123"  
"98765"
```

Por ejemplo:

`[a-zA-Z_][a-zA-Z0-9_]*`

permite que después del primer carácter de un identificador aparezca cualquier cantidad de letras, números o guiones bajos.

1.2.8. Una o más repeticiones

El operador `+` significa que el elemento anterior debe aparecer una o más veces.

`[0-9] +`

coincide con:

```
1  
123  
98765
```

pero no coincide con una cadena vacía.

Por ejemplo, esta es una forma habitual de reconocer números enteros:

```
[0-9]+ { return NUMERO; }
```

1.2.9. Cero o una aparición

El operador `?` indica que el elemento anterior puede aparecer cero o una vez.

`-?[0-9]+`

permite reconocer:

```
123  
-123  
45  
-45
```

Por lo tanto, puede utilizarse para representar un signo negativo opcional.

1.2.10. Repeticiones exactas y rangos

Las expresiones entre llaves permiten especificar cantidades de repeticiones.

`[0-9]{4}`

indica exactamente cuatro dígitos.

Por ejemplo:

2026

1234

9876

También pueden especificarse rangos:

`[0-9]{2,4}`

que permite entre dos y cuatro dígitos.

`[0-9]{2,}`

permite dos o más dígitos.

1.2.11. Cualquier carácter

El punto `.` representa cualquier carácter excepto, en general, un salto de línea.

Por ejemplo:

```
. { return CHARACTER; }
```

puede utilizarse como regla general para detectar caracteres que no fueron reconocidos por reglas anteriores.

Esta característica resulta especialmente útil para detectar errores léxicos:

```
. {  
    printf("Error léxico: %s\n", yytext);  
}
```

1.2.12. Paréntesis y agrupación

Los paréntesis permiten agrupar patrones para aplicar operadores sobre varios elementos.

Por ejemplo:

`(ab)+`

coincide con:

ab

abab

ababab

También pueden combinarse con alternancia:

`(if|while|for)`

1.2.13. Caracteres especiales

Algunos caracteres poseen un significado especial dentro de una expresión regular. Entre ellos se encuentran:

`. * + ? | ( ) [ ] { } ^ $`

Cuando se necesita reconocer literalmente uno de estos caracteres, puede utilizarse una barra inversa para escaparlo.

Por ejemplo:

```
\+
\\*
\\(
\\)
\\[
\\]
\\{
\\}
\\.

```

También es habitual utilizar comillas para representar operadores como literales:

```
"+"      { return SUMA; }
"*"      { return MULTIPLICACION; }
"("      { return PARENTESIS_IZQ; }
")"      { return PARENTESIS_DER; }
```

1.2.14. Caracteres especiales de escape

Flex permite utilizar secuencias de escape para representar caracteres especiales.

Algunos de los más utilizados son:

Secuencia	Significado
<code>\n</code>	salto de línea
<code>\t</code>	tabulación
<code>\r</code>	retorno de carro
<code>\f</code>	salto de página
<code>\v</code>	tabulación vertical
<code>\\</code>	barra inversa

Por ejemplo:

```
[ \t\n]+ { /* ignorar espacios */ }
```

1.2.15. Inicio y final de línea

El carácter `^` fuera de una clase de caracteres puede utilizarse para indicar el comienzo de una línea.

Por ejemplo:

```
^int
```

coincide con `int` cuando aparece al comienzo de una línea.

El carácter `$` puede utilizarse para indicar el final de una línea.

Por ejemplo:

```
int$
```

coincide con `int` cuando aparece al final de una línea.

1.2.16. Contexto posterior

Flex permite utilizar el operador / para especificar contexto posterior.
Por ejemplo:

`[0-9]+/[a-zA-Z]`

indica que una secuencia de números debe estar seguida por una letra para que el patrón sea considerado un emparejamiento válido.

El texto utilizado como contexto posterior participa en la determinación del emparejamiento, aunque no necesariamente forma parte del texto devuelto como token.

1.2.17. Macros de patrones

Flex permite definir nombres para patrones en la sección de definiciones. Esto evita repetir expresiones regulares complejas.

Por ejemplo:

```
DIGITO      [0-9]
LETRA       [a-zA-Z]
ID          {LETRA}({LETRA}|{DIGITO})*
```

Luego pueden utilizarse estos nombres en las reglas:

```
%%
{ID}        { return IDENTIFICADOR; }
{DIGITO}+   { return NUMERO; }
```

Esto permite construir patrones complejos a partir de patrones más simples y hace que la especificación sea más legible.

1.3. Resumen de patrones

Los principales operadores de expresiones regulares utilizados en Flex pueden resumirse de la siguiente manera:

Patrón	Significado
<code>a</code>	carácter literal
<code>"abc"</code>	cadena literal
<code>[abc]</code>	uno de los caracteres indicados
<code>[a-z]</code>	un carácter dentro de un rango
<code>[^a-z]</code>	cualquier carácter fuera del rango
<code>.</code>	cualquier carácter excepto salto de línea
<code>ab</code>	concatenación de patrones
<code>a b</code>	alternativa entre patrones
<code>a*</code>	cero o más repeticiones
<code>a+</code>	una o más repeticiones
<code>a?</code>	cero o una aparición
<code>a{n}</code>	exactamente n repeticiones
<code>a{n,m}</code>	entre n y m repeticiones
<code>(ab)</code>	agrupación
<code>^a</code>	comienzo de línea
<code>a\$</code>	final de línea
<code>a/b</code>	contexto posterior
<code>\c</code>	carácter especial escapado

Estos patrones constituyen las herramientas fundamentales para describir los lexemas de un lenguaje de programación.

1.4. Proceso de emparejamiento

Cuando el escáner generado por Flex se encuentra en una determinada posición de la entrada, busca los patrones que pueden comenzar en dicha posición. Cada patrón intenta consumir la mayor cantidad de caracteres posible.

Por ejemplo, si se definen las siguientes reglas:

```
[0-9]      { return DIGITO; }  
[0-9]+     { return NUMERO; }
```

y la entrada es:

1234 + 1

al comenzar en el primer carácter, ambas reglas pueden coincidir:

Patrón	Emparejamiento
[0-9]	1
[0-9]+	1234

Flex selecciona el emparejamiento que consume más caracteres. Por lo tanto, selecciona:

1234 $\longrightarrow$ NUMERO

y no:

1 $\longrightarrow$ DIGITO

La posición de análisis avanza solamente después de haber determinado el emparejamiento ganador. En este caso, después de reconocer 1234, la entrada restante es:

+ 1

A continuación, Flex repite el mismo proceso desde la nueva posición.

De esta manera, el proceso puede resumirse en los siguientes pasos:

1. Se fija la posición actual de la entrada.
2. Se buscan todos los patrones que pueden coincidir desde esa posición.
3. Cada patrón intenta extender el emparejamiento tanto como sea posible.
4. Se conservan los emparejamientos válidos encontrados.
5. Se selecciona el emparejamiento que consume la mayor cantidad de caracteres.
6. Se ejecuta la acción asociada al patrón seleccionado.
7. La posición de la entrada avanza hasta el final del lexema reconocido.
8. El proceso se repite hasta alcanzar el final de la entrada.

Este comportamiento se conoce como el principio de *emparejamiento más largo* o *maximal munch*.

Si dos o más patrones producen emparejamientos de la misma longitud, Flex selecciona el patrón que aparece primero en el archivo de especificación.

1.5. Flex como productor de tokens

Cuando Flex se utiliza como parte de un compilador, las acciones de las reglas pueden utilizarse para producir tokens que posteriormente serán consumidos por el parser generado por Bison.

Por ejemplo:

```
%%
"int"                { return INT; }
[0-9]+               { return NUMERO; }
[a-zA-Z_][a-zA-Z0-9_]* { return IDENTIFICADOR; }
"+"                 { return SUMA; }
"="                 { return ASIGNACION; }
";"                 { return PUNTO_COMA; }
[ \t\n]+             { /* ignorar */ }
```

Para una entrada como:

```
int x = 10 + 20;
```

el análisis léxico produce conceptualmente:

```
[INT, IDENTIFICADOR, ASIGNACION, NUMERO, SUMA, NUMERO, PUNTO_COMA]
```

Los espacios, tabulaciones y saltos de línea pueden ser reconocidos por una regla específica y descartados mediante una acción vacía.

1.6. Lexema reconocido y variables de Flex

Una vez determinado el emparejamiento, Flex pone a disposición de la acción correspondiente el texto reconocido mediante la variable `yytext` y su longitud mediante la variable global entera `yylen`.

Por ejemplo:

```
[0-9]+ {
    printf("Lexema: %s\n", yytext);
    printf("Longitud: %d\n", yylen);
}
```

Si la entrada contiene:

```
1234
```

el valor de `yytext` será:

```
1234
```

mientras que `yylen` tendrá el valor:

4

Esto permite que la acción asociada a una regla trabaje con el lexema completo que fue reconocido y no solamente con el primer carácter que coincidió con el patrón.

1.7. Camino no feliz

Un caso no feliz ocurre cuando el código fuente contiene un carácter o secuencia que no coincide con ninguno de los patrones definidos.

Por ejemplo:

```
int x = 10 @ 20;
```

Si el lenguaje no define el carácter @, podemos utilizar una regla general para detectarlo:

```
. {  
    printf("Error léxico: %s\n", yytext);  
}
```

El procesamiento será conceptualmente:

Lexema	Resultado
int	INT
x	IDENTIFICADOR
=	ASIGNACION
10	NUMERO
@	ERROR LÉXICO
20	NUMERO
;	PUNTO_COMA

Por lo tanto, Flex recorre la entrada buscando continuamente el emparejamiento más largo disponible en cada posición. Una vez reconocido el lexema, ejecuta la acción correspondiente y continúa el análisis desde el carácter siguiente.

Finalmente, el flujo completo puede resumirse como:

Código fuente $\xrightarrow{\text{Flex}}$ Secuencia de tokens $\xrightarrow{\text{Bison}}$ Análisis sintáctico